DBMS – WEEK 10 REPORT

SAREE BOUTIQUE MANAGEMENT SYSTEM

Advanced SQL Query System

Scenario

The Saree Boutique Management System manages saree products, customer orders, product categories, and sales details. The management team wants to perform advanced analysis on customer purchases, product pricing, and sales performance.

SQL Subqueries and Nested Queries are used to identify premium saree products, high-value customers, best-selling products, category-wise sales, and business trends. These queries help the boutique management make better business decisions.

Aim

To implement Subqueries and Nested Queries in the Saree Boutique Management System to analyze product pricing, customer purchases, and sales performance.

Objectives

1. To implement Single-row, Multi-row, and Nested Subqueries.
2. To identify products priced above average price.
3. To identify customers with maximum purchases.
4. To analyze best-selling products and category performance.
5. To generate advanced business reports for decision-making.

---

1. Implement Subqueries and Nested Queries Concept

A Subquery is a query written inside another SQL query. It helps to retrieve data using the result of another query.

Types of Subqueries

1. Single-row Subquery
2. Multi-row Subquery
3. Nested Subquery

1.1 Single-row Subquery

Find saree products whose price is greater than the average product price.

Step 1: Find Average Price

```
SQL> SELECT AVG(PRICE)
  2  FROM PRODUCT1;

AVG(PRICE)
----------
      2261
```

Step 2: Find Products Above Average Price

```
SQL> SELECT PRODUCT_NAME, PRICE
  2  FROM PRODUCT1
  3  WHERE PRICE > (
  4      SELECT AVG(PRICE)
  5      FROM PRODUCT1
  6  );

PRODUCT_NAME
--------------------------------------------------------------------
--
     PRICE
----------
Kanchipuram Silk Saree
      8500
```

2. Find Products Above Average Price

Scenario:

The Saree Boutique wants to identify premium saree products based on their prices. Subqueries are used to compare product prices with the average product price and category-wise average price.

Task 1: Find Products Priced Above the Average Price

```
SQL> SELECT PRODUCT_ID,
  2         PRODUCT_NAME,
  3         PRICE
  4  FROM PRODUCT1
  5  WHERE PRICE > (
  6      SELECT AVG(PRICE)
  7      FROM PRODUCT1
  8  );

PRODUCT
-------
PRODUCT_NAME
----------------------------------------
--
     PRICE
----------
P001
Kanchipuram Silk Saree
      8500
```

Task 2: Display Product Name, Category, and Price

```
SQL> SELECT
  2      P.PRODUCT_NAME,
  3      C.CATEGORY_NAME,
  4      P.PRICE
  5  FROM PRODUCT1 P
  6  JOIN CATEGORY1 C
  7      ON P.CATEGORY_ID = C.CATEGORY_ID
  8  WHERE P.PRICE > (
  9      SELECT AVG(PRICE)
 10      FROM PRODUCT1
 11  );

PRODUCT_NAME
----------------------------------------
--
CATEGORY_NAME             PRICE
-------------------- ----------
Kanchipuram Silk Saree
Kanchipuram Silk           8500

Kanchipuram Silk Saree
Kanchipuram Silk           8500

Kanchipuram Silk Saree
Kanchipuram Silk           8500
```

Task 3: Identify Expensive Products in Each Category

```
SQL> SELECT
  2      P.PRODUCT_ID,
  3      P.PRODUCT_NAME,
  4      P.CATEGORY_ID,
  5      P.PRICE
  6  FROM PRODUCT1 P
  7  WHERE P.PRICE > (
  8      SELECT AVG(P2.PRICE)
  9      FROM PRODUCT1 P2
 10      WHERE P2.CATEGORY_ID = P.CATEGORY_ID
 11  );

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------
--
CATEGOR      PRICE
------- ----------
P101
Shoe
CAT101         378
```

Task 4: Find Products with Price Higher Than a Selected Category Average

```
SQL> SELECT PRODUCT_ID,
  2         PRODUCT_NAME,
  3         PRICE
  4  FROM PRODUCT1
  5  WHERE PRICE > (
  6      SELECT AVG(PRICE)
  7      FROM PRODUCT1
  8      WHERE CATEGORY_ID = 'SC001'
  9  );

no rows selected
```

---

3. Identify Customers with Maximum Purchases

Task 1: Find the Customer Who Spent the Highest Amount

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      CUSTOMER_NAME,
  4      SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
  5  FROM ORDERS
  6  GROUP BY CUSTOMER_ID, CUSTOMER_NAME
  7  HAVING SUM(TOTAL_AMOUNT) = (
  8      SELECT MAX(TOTAL_SALES)
  9      FROM (
 10          SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SALES
 11          FROM ORDERS
 12          GROUP BY CUSTOMER_ID
 13      )
 14  );

CUSTOME CUSTOMER_NAME                                    TOTAL_SPENDING
------- ------------------------------------------------ --------------
CUS0001 YAMINI                                                    25000

SQL>
```

Task 2: Find Customers with Maximum Number of Orders

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      COUNT(ORDER_ID) AS NUMBER_OF_ORDERS
  4  FROM ORDERS
  5  GROUP BY CUSTOMER_ID
  6  HAVING COUNT(ORDER_ID) = (
  7      SELECT MAX(ORDER_COUNT)
  8      FROM (
  9          SELECT COUNT(ORDER_ID) AS ORDER_COUNT
 10          FROM ORDERS
 11          GROUP BY CUSTOMER_ID
 12      )
 13  );

CUSTOME NUMBER_OF_ORDERS
------- ----------------
CUS0002                1
CUS0001                1
```

Task 3: Find Customers Whose Spending Is Above Average

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      CUSTOMER_NAME,
  4      SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
  5  FROM ORDERS
  6  GROUP BY CUSTOMER_ID, CUSTOMER_NAME
  7  HAVING SUM(TOTAL_AMOUNT) > (
  8      SELECT AVG(TOTAL_SPENDING)
  9      FROM (
 10          SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
 11          FROM ORDERS
 12          GROUP BY CUSTOMER_ID
 13      )
 14  );

CUSTOME CUSTOMER_NAME                                    TOTAL_SPENDING
------- ------------------------------------------------ --------------
CUS0001 YAMINI                                                    25000
```

Task 4: Find Top 5 Valuable Customers

```
SQL> SELECT *
  2  FROM (
  3      SELECT
  4          CUSTOMER_ID,
  5          CUSTOMER_NAME,
  6          SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
  7      FROM ORDERS
  8      GROUP BY CUSTOMER_ID, CUSTOMER_NAME
  9      ORDER BY TOTAL_SPENDING DESC
 10  )
 11  WHERE ROWNUM <= 5;

CUSTOME CUSTOMER_NAME                                    TOTAL_SPENDING
------- ------------------------------------------------ --------------
CUS0001 YAMINI                                                    25000
CUS0002 NANDY                                                     15000
```

---

4. Generate Complex Business Queries

Scenario:

The Saree Boutique management wants to generate advanced business reports to understand product sales, customer value, category performance, and customer purchase history.

Query 1: Best-Selling Product

Task: Find products with maximum sales quantity and display product name, total quantity sold, and total revenue generated.

```
SQL> SELECT
  2      P.PRODUCT_NAME,
  3      SUM(OD.QUANTITY) AS TOTAL_QUANTITY_SOLD,
  4      SUM(P.PRICE * OD.QUANTITY) AS TOTAL_REVENUE
  5  FROM PRODUCT1 P
  6  JOIN ORDER_DETAILS OD
  7      ON P.PRODUCT_ID = OD.PRODUCT_ID
  8  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  9  HAVING SUM(OD.QUANTITY) = (
 10      SELECT MAX(TOTAL_QUANTITY)
 11      FROM (
 12          SELECT SUM(QUANTITY) AS TOTAL_QUANTITY
 13          FROM ORDER_DETAILS
 14          GROUP BY PRODUCT_ID
 15      )
 16  );

PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------- -------------
Shoe
                  2           756

T-Shirt
                  2          1198

Cotton Saree
                  2          3600
```

Query 2: High-Value Customers

Task: Find customers whose total purchase amount is greater than average customer spending.

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      CUSTOMER_NAME,
  4      SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
  5  FROM ORDERS
  6  GROUP BY CUSTOMER_ID, CUSTOMER_NAME
  7  HAVING SUM(TOTAL_AMOUNT) > (
  8      SELECT AVG(TOTAL_SPENDING)
  9      FROM (
 10          SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
 11          FROM ORDERS
 12          GROUP BY CUSTOMER_ID
 13      )
 14  );

CUSTOME CUSTOMER_NAME                                      TOTAL_SPENDING
------- -------------------------------------------------- --------------
CUS0001 YAMINI                                                      25000
```

Query 3: Category Performance Analysis

Task 1: Find the Highest Revenue-Generating Category

```
SQL> SELECT *
  2  FROM (
  3      SELECT
  4          C.CATEGORY_ID,
  5          C.CATEGORY_NAME,
  6          SUM(P.PRICE * OD.QUANTITY) AS TOTAL_REVENUE
  7      FROM CATEGORY1 C
  8      JOIN PRODUCT1 P
  9          ON C.CATEGORY_ID = P.CATEGORY_ID
 10      JOIN ORDER_DETAILS OD
 11          ON P.PRODUCT_ID = OD.PRODUCT_ID
 12      GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME
 13      ORDER BY TOTAL_REVENUE DESC
 14  )
 15  WHERE ROWNUM = 1;

CATEGOR CATEGORY_NAME        TOTAL_REVENUE
------- -------------------- -------------
SC001   Kanchipuram Silk             25500
```

Task 2: Find the Number of Products Sold in Each Category

```
SQL> SELECT
  2      C.CATEGORY_ID,
  3      C.CATEGORY_NAME,
  4      NVL(SUM(OD.QUANTITY), 0) AS TOTAL_PRODUCTS_SOLD
  5  FROM CATEGORY1 C
  6  JOIN PRODUCT1 P
  7      ON C.CATEGORY_ID = P.CATEGORY_ID
  8  LEFT JOIN ORDER_DETAILS OD
  9      ON P.PRODUCT_ID = OD.PRODUCT_ID
 10  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME
 11  ORDER BY TOTAL_PRODUCTS_SOLD DESC;

CATEGOR CATEGORY_NAME        TOTAL_PRODUCTS_SOLD
------- -------------------- -------------------
SC001   Kanchipuram Silk                       3
 SC002  Cotton Sarees                          2
```

Task 3: Calculate Average Product Price by Category

```
SQL> SELECT
  2      C.CATEGORY_ID,
  3      C.CATEGORY_NAME,
  4      AVG(P.PRICE) AS AVERAGE_PRODUCT_PRICE
  5  FROM CATEGORY1 C
  6  JOIN PRODUCT1 P
  7      ON C.CATEGORY_ID = P.CATEGORY_ID
  8  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME;

CATEGOR CATEGORY_NAME        AVERAGE_PRODUCT_PRICE
------- -------------------- ---------------------
 SC002  Cotton Sarees                         1800
SC001   Kanchipuram Silk                      8500
```

Query 4: Customer Purchase History

Task: Display customer name, number of orders, total purchase amount, and most purchased product.

```
SQL> WITH CUSTOMER_TOTALS AS (
  2      SELECT
  3          CUSTOMER_ID,
  4          CUSTOMER_NAME,
  5          COUNT(ORDER_ID) AS NUMBER_OF_ORDERS,
  6          SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
  7      FROM ORDERS
  8      GROUP BY CUSTOMER_ID, CUSTOMER_NAME
  9  ),
 10  PRODUCT_RANKING AS (
 11      SELECT
 12          O.CUSTOMER_ID,
 13          P.PRODUCT_NAME,
 14          SUM(OD.QUANTITY) AS TOTAL_QUANTITY,
 15          ROW_NUMBER() OVER (
 16              PARTITION BY O.CUSTOMER_ID
 17              ORDER BY SUM(OD.QUANTITY) DESC
 18          ) AS RN
 19      FROM ORDERS O
 20      JOIN ORDER_DETAILS OD
 21          ON O.ORDER_ID = OD.ORDER_ID
 22      JOIN PRODUCT1 P
 23          ON OD.PRODUCT_ID = P.PRODUCT_ID
 24      GROUP BY O.CUSTOMER_ID, P.PRODUCT_NAME
 25  )
 26  SELECT
 27      C.CUSTOMER_NAME,
 28      C.NUMBER_OF_ORDERS,
 29      C.TOTAL_PURCHASE,
 30      P.PRODUCT_NAME AS MOST_PURCHASED_PRODUCT
 31  FROM CUSTOMER_TOTALS C
 32  LEFT JOIN PRODUCT_RANKING P
 33      ON C.CUSTOMER_ID = P.CUSTOMER_ID
 34      AND P.RN = 1;

CUSTOMER_NAME                                    NUMBER_OF_ORDERS
------------------------------------------------ ----------------
TOTAL_PURCHASE
--------------
MOST_PURCHASED_PRODUCT
--------------------------------------------------------------------------------
YAMINI                                                          1
        25000
Shoe

NANDY                                                           1
        15000
Cotton Saree

CUSTOMER_NAME                                    NUMBER_OF_ORDERS
------------------------------------------------ ----------------
TOTAL_PURCHASE
--------------
MOST_PURCHASED_PRODUCT
--------------------------------------------------------------------------------
```

---

5. Prepare Advanced SQL Report

Scenario:

The Saree Boutique management wants to prepare advanced reports to identify premium products, valuable customers, sales performance, and business trends. These reports help the boutique understand its performance and make better business decisions.

Report 1: Premium Product Report

Task: Display product name, category, price, and stock availability condition for products above average price.

```
SQL> SELECT
  2      P.PRODUCT_NAME,
  3      C.CATEGORY_NAME,
  4      P.PRICE,
  5      P.STOCK_QUANTITY,
  6      CASE
  7          WHEN P.STOCK_QUANTITY > 0
  8              THEN 'In Stock'
  9          ELSE 'Out of Stock'
 10      END AS STOCK_CONDITION
 11  FROM PRODUCT1 P
 12  JOIN CATEGORY1 C
 13      ON P.CATEGORY_ID = C.CATEGORY_ID
 14  WHERE P.PRICE > (
 15      SELECT AVG(PRICE)
 16      FROM PRODUCT1
 17  );

PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGORY_NAME             PRICE STOCK_QUANTITY STOCK_CONDIT
-------------------- ---------- -------------- ------------
Kanchipuram Silk Saree
Kanchipuram Silk           8500             10 In Stock

Kanchipuram Silk Saree
Kanchipuram Silk           8500             10 In Stock

Kanchipuram Silk Saree
Kanchipuram Silk           8500             10 In Stock
```

Report 2: Customer Value Report

Task: Display customer name, total orders, total spending, and customer category.

```
SQL> WITH CUSTOMER_VALUE AS (
  2      SELECT
  3          CUSTOMER_ID,
  4          CUSTOMER_NAME,
  5          COUNT(ORDER_ID) AS TOTAL_ORDERS,
  6          SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
  7      FROM ORDERS
  8      GROUP BY CUSTOMER_ID, CUSTOMER_NAME
  9  )
 10  SELECT
 11      CUSTOMER_NAME,
 12      TOTAL_ORDERS,
 13      TOTAL_SPENDING,
 14      CASE
 15          WHEN TOTAL_SPENDING >= (
 16              SELECT AVG(TOTAL_SPENDING)
 17              FROM CUSTOMER_VALUE
 18          )
 19          THEN 'Premium'
 20          ELSE 'Regular'
 21      END AS CUSTOMER_CATEGORY
 22  FROM CUSTOMER_VALUE
 23  ORDER BY TOTAL_SPENDING DESC;

CUSTOMER_NAME                                      TOTAL_ORDERS TOTAL_SPENDING
-------------------------------------------------- ------------ --------------
CUSTOME
-------
YAMINI                                                        1          25000
Premium

NANDY                                                         1          15000
Regular
```

Report 3: Sales Performance Report

Task 1: Product-wise Sales

```
SQL> SELECT
  2      P.PRODUCT_ID,
  3      P.PRODUCT_NAME,
  4      NVL(SUM(OD.QUANTITY), 0) AS QUANTITY_SOLD,
  5      NVL(SUM(P.PRICE * OD.QUANTITY), 0) AS REVENUE_GENERATED
  6  FROM PRODUCT1 P
  7  LEFT JOIN ORDER_DETAILS OD
  8      ON P.PRODUCT_ID = OD.PRODUCT_ID
  9  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
 10  ORDER BY REVENUE_GENERATED DESC;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD REVENUE_GENERATED
------------- -----------------
P001
Kanchipuram Silk Saree
            1              8500

P002
Cotton Saree
            2              3600

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD REVENUE_GENERATED
------------- -----------------

P102
T-Shirt
            2              1198

P101
Shoe

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD REVENUE_GENERATED
------------- -----------------
            2               756

P111
Sketch
            1                28
```

Task 2: Category-wise Revenue

```
SQL> SELECT
  2      C.CATEGORY_NAME,
  3      SUM(P.PRICE * OD.QUANTITY) AS TOTAL_REVENUE
  4  FROM CATEGORY1 C
  5  JOIN PRODUCT1 P
  6      ON C.CATEGORY_ID = P.CATEGORY_ID
  7  JOIN ORDER_DETAILS OD
  8      ON P.PRODUCT_ID = OD.PRODUCT_ID
  9  GROUP BY C.CATEGORY_NAME
 10  ORDER BY TOTAL_REVENUE DESC;

CATEGORY_NAME        TOTAL_REVENUE
-------------------- -------------
Kanchipuram Silk             25500
Cotton Sarees                 3600
```

Task 3: Top-Selling Products

```
SQL> SELECT *
  2  FROM (
  3      SELECT
  4          P.PRODUCT_ID,
  5          P.PRODUCT_NAME,
  6          SUM(OD.QUANTITY) AS TOTAL_QUANTITY_SOLD
  7      FROM PRODUCT1 P
  8      JOIN ORDER_DETAILS OD
  9          ON P.PRODUCT_ID = OD.PRODUCT_ID
 10      GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
 11      ORDER BY TOTAL_QUANTITY_SOLD DESC
 12  )
 13  WHERE ROWNUM <= 5;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
P101
Shoe
                  2

P102
T-Shirt
                  2

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------

P002
Cotton Saree
                  2

P111
Sketch

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
                  1

P001
Kanchipuram Silk Saree
                  1
```

Report 4: Business Decision Report

Task 1: Identify High-Performing Products

```
SQL> SELECT
  2      P.PRODUCT_ID,
  3      P.PRODUCT_NAME,
  4      SUM(OD.QUANTITY) AS TOTAL_QUANTITY_SOLD,
  5      SUM(P.PRICE * OD.QUANTITY) AS TOTAL_REVENUE
  6  FROM PRODUCT1 P
  7  JOIN ORDER_DETAILS OD
  8      ON P.PRODUCT_ID = OD.PRODUCT_ID
  9  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
 10  ORDER BY TOTAL_REVENUE DESC;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------- -------------
P001
Kanchipuram Silk Saree
                  1          8500

P002
Cotton Saree
                  2          3600

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------- -------------

P102
T-Shirt
                  2          1198

P101
Shoe

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------- -------------
                  2           756

P111
Sketch
                  1            28
```

Task 2: Identify Low-Performing Products

```
SQL> SELECT
  2      P.PRODUCT_ID,
  3      P.PRODUCT_NAME,
  4      NVL(SUM(OD.QUANTITY), 0) AS TOTAL_QUANTITY_SOLD
  5  FROM PRODUCT1 P
  6  LEFT JOIN ORDER_DETAILS OD
  7      ON P.PRODUCT_ID = OD.PRODUCT_ID
  8  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  9  ORDER BY TOTAL_QUANTITY_SOLD ASC;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
P111
Sketch
                  1

P001
Kanchipuram Silk Saree
                  1

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------

P102
T-Shirt
                  2

P101
Shoe

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
                  2

P002
Cotton Saree
                  2
```

Task 3: Identify High-Value Customers

```
SQL> SELECT
  2      CUSTOMER_ID,
  3      CUSTOMER_NAME,
  4      SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
  5  FROM ORDERS
  6  GROUP BY CUSTOMER_ID, CUSTOMER_NAME
  7  HAVING SUM(TOTAL_AMOUNT) > (
  8      SELECT AVG(TOTAL_SPENDING)
  9      FROM (
 10          SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
 11          FROM ORDERS
 12          GROUP BY CUSTOMER_ID
 13      )
 14  )
 15  ORDER BY TOTAL_SPENDING DESC;

CUSTOME CUSTOMER_NAME                                      TOTAL_SPENDING
------- -------------------------------------------------- --------------
CUS0001 YAMINI                                                      25000
```

Task 4: Analyze Monthly Revenue Trends

```
SQL> SELECT
  2      TO_CHAR(ORDER_DATE, 'YYYY-MM') AS SALES_MONTH,
  3      SUM(TOTAL_AMOUNT) AS MONTHLY_REVENUE
  4  FROM ORDERS
  5  GROUP BY TO_CHAR(ORDER_DATE, 'YYYY-MM')
  6  ORDER BY SALES_MONTH;

SALES_M MONTHLY_REVENUE
------- ---------------
2026-09           40000
```

Conclusion

The Saree Boutique Management System uses advanced SQL Subqueries and Nested Queries to analyze product prices, customer purchases, and sales performance. These queries help identify premium sarees, valuable customers, best-selling products, and category-wise revenue. The generated reports support better business decisions and improve the overall management of the boutique.